

Audit complet -- opti_route au 2026-05-10

Audit réalisé après les 13 PRs de cette session (PR #55 -> #67), 68 PRs au total depuis le début du projet.

Résumé exécutif

****Verdict global : ~80 % prêt pour publication Play Store.****

L'app couvre fonctionnellement l'intégralité du cycle de vie d'une tournée de livraison (création -> géocodage -> optimisation -> exécution avec GPS -> validation -> bilan -> archivage). Stack 100 % gratuite respectée, aucune carte de crédit requise.

Ce qui manque avant publication : génération effective de la keystore release, screenshots Play Store, compte développeur Google (25 \$ unique, là il faudra une CB pour Google), tests sur appareils Android variés.

1. Volumes & métriques

Métrique	Valeur
PRs mergées (total)	**68**
Fichiers Dart (hors généré)	**51**
Lignes de code Dart	**12 633**
Lignes de tests	**1 943**
Tests unitaires qui passent	**85 / 85**
Issues `flutter analyze`	**0**
`TODO` / `FIXME` dans le code	**0**
Dépendances pubspec	11 (dont 2 dev)

****Ratio tests/code : 15 %****. Honnête pour ce type de projet. Les zones critiques sont bien couvertes :

- * Parser bordereau OCR (cas réels avec données ML Kit)
- * Service optimisation (cascade firsts/flexibles/lasts)
- * Repos Drift (CRUD + carnet + duplication)
- * Service géocodage SIRENE (filtres etat_administratif)

Pas de tests d'****intégration UI**** (widget tests light, pas de golden tests). Acceptable pour la phase actuelle, sera utile avant la prod publique.

2. Structure du code

```
app/lib/  
??? data/          25 fichiers -- services, repos, modèles  
??? providers/      5 fichiers -- providers Riverpod  
??? screens/        14 fichiers -- écrans  
??? widgets/        5 fichiers -- composants réutilisables  
??? theme/          2 fichiers -- tokens design + thème Material
```

Architecture claire ****3 couches**** : `data` (logique métier) <- `providers` (DI) <- `screens` / `widgets` (UI). Pas de fuite vers le bas, pas de dépendance circulaire détectée.

****Cohérence des conventions**** :

- * Tous les commits suivent Conventional Commits

- * Tous les merges sont en squash (1 PR = 1 commit sur main)
- * Tous les écrans suivent le même pattern Material 3
- * Tous les services exposent `close()` pour le cleanup

3. Stack technique

Production (10 deps)

Dep	Version	Usage
flutter_riverpod	3.3.1	State management
drift + drift_flutter	2.33	Base SQLite locale
http	1.6	APIs externes (BAN, ORS, SIRENE, Photon)
flutter_map + latlong2	8.3 + 0.9	
google_mlkit_text_recognition	0.15	OCR caméra
image_picker	1.2	Capture photo bordereau
url_launcher	6.3	Lancement Maps/Waze
geolocator	14.0	GPS temps réel
share_plus + path_provider	11 + 2	Export CSV/PDF
pdf	3.11	Génération PDF tournée

APIs externes (toutes gratuites)

API	Quota	CB requise
BAN (api-adresse.data.gouv.fr)	usage raisonnable	non
Recherche-Entreprises (SIRENE)	usage raisonnable	non
Photon (Komoot)	usage raisonnable	non
OpenRouteService	500 optim/j + 2000 directions/j	non
Tuiles OSM	usage raisonnable	non

****Strictement free**** : confirmé pour toute la stack.

4. Features livrées (28 axes)

Setup & socle (#1 -> #5)

- * Pre-push hook bloque push direct sur main
- * DB Drift avec schemaVersion 8
- * Thème Material 3 custom (cream/ink/lime/emerald + Manrope + JetBrains Mono)

Géocodage (#6 -> #23, #41, #42)

- * Cascade BAN -> SIRENE -> Photon optimisée pour la France
- * Filtre des entreprises SIRENE cessées (état_administratif = 'C' / 'F')
- * Cache local des résultats
- * Adresse postale nettoyée (sans préfixe nom)
- * Géocodage manuel par tap sur la carte (avec reverse geocoding BAN)

Tournée & arrêts (#7 -> #51, #56 -> #58, #63)

- * CRUD complet
- * Validation Livré / Échec (avec raison)

- * Priorités EN 1ER / EN DERNIER avec drag-and-drop d'ordre
- * Drag-and-drop manuel des arrêts dans la tournée
- * Recherche dans la liste arrêts (5+ arrêts)
- * Multi-tournées par jour avec bandeau de switch
- * Templates de tournée (duplication avec reset des statuts)
- * Stats cumulatives 7/30/365 j

Mode terrain (#39, #40, #45, #55, #61)

- * GPS temps réel avec carte "Prochain arrêt"
- * Boutons Maps / Waze rapides (avec préférence par défaut)
- * Bottom sheet d'arrêt avec stepper colis + Maps/Waze + Livré/Échec
- * Badge "tournée en cours" globale sur l'icône drawer

Optimisation (#18, #37, #38, #44, #52)

- * VROOM via OpenRouteService
- * Distance/durée totales sur tournée complète (`/directions/geojson`)
- * Tracé polyline sur la carte
- * Bouton Optimiser grisé tant que rien n'a changé

OCR bordereaux MESEXP (#25 -> #35)

- * Parser robuste basé sur les vraies données ML Kit
- * Score de confiance + carte orange pour bordereaux ambigus
- * Filtres anti-rues numérotées, anti-transporteurs

Carnet d'adresses local (#36, #48, #58)

- * Auto-mémorisation à chaque arrêt validé
- * Édition manuelle + recherche tolérante aux accents
- * Export CSV partageable

Polish & publication (#53, #62, #64 -> #67)

- * App icon + splash screen finalisés
- * Onboarding 3 pages au premier lancement
- * Export PDF récap de tournée
- * CI GitHub Actions (analyze + test sur PR)
- * Mentions légales (privacy + CGU)
- * Signing config release (procédure documentée)

5. Tests -- couverture qualitative

Domaine	Tests	Qualité
`BordereauParser`	34 tests	***** excellents -- basés sur vraies données ML Kit terrain
`OpenRouteOptimizationService`	7 tests	**** couvre les cas firsts/lasts + fallback haversine
`StatsService`	4 tests	**** agrégations + bornes
`RechercheEntreprisesService`	4 tests	**** filtres etat_administratif
`SavedDestinationsRepository`	6 tests	**** upsert + recherche accents
`StopsRepository`	5 tests	*** statuts livraison
`TournéesRepository`	6 tests	**** duplication + reset
`CarnetExportService`	3 tests	*** format CSV RFC 4180
`NavigationService`	3 tests	** URIs Maps / Waze

`AddressSuggestion`	4 tests	*** adressePostale propre
`AppDatabase`	4 tests	** cascade delete

****Trou de couverture** :**

- * Widgets (UI) : aucun test
 - * `OcrService`, `LocationService` : pas de test (difficile sans bind plateforme)
 - * `TournéePdfService` : pas de test (idem)
- > Pas critique tant que la base reste petite et qu'on teste à la main sur l'appareil.

6. Zones de fragilité identifiées

6.1 Mode sombre absent (reporté)

Le thème actuel utilise `AppColors.\*` en dur partout (50+ occurrences). Bascule en mode sombre demanderait de tout passer par `Theme.of(context).colorScheme.\*`. Refactor de plusieurs heures.

****Impact**** : conduite de nuit possible mais éblouissante. Pas bloquant.

6.2 Mode hors-ligne absent

Si Noah perd la 4G en zone rurale :

- * Géocodage ne fonctionne pas (les recherches d'adresse échouent)
- * Tuiles OSM non cachées -> carte blanche
- * ORS optimisation ne tourne pas

****Impact**** : terrain. Le carnet d'adresses local sauve la mise pour les clients connus, mais une nouvelle adresse ne sera pas géocodable hors-ligne.

6.3 Tests UI absents

Si un refactor du `TournéeDuJourScreen` casse l'affichage sans casser la logique, `flutter analyze` + `flutter test` ne le détectera pas.

****Impact**** : limité tant qu'on teste à la main sur l'appareil après chaque PR.

6.4 Signing config debug par défaut

Tant que `key.properties` n'est pas généré, l'APK release est signé avec les clés debug. Non publiable sur Play Store.

****Impact**** : bloque la publication mais le wiring Gradle est prêt. Procédure documentée dans `docs/keystore-release.md`.

6.5 PDF du journal régénéré à chaque commit

Le binaire change à chaque exécution de `\_build\_pdf.py` (timestamp interne ?). Pas un vrai problème mais fait toujours apparaître un "uncommitted change" après chaque PR.

****Impact**** : cosmétique. À examiner plus tard.

7. Risques sécurité & RGPD

- * ? ****Aucune donnée personnelle envoyée à un serveur tiers contrôlé par l'auteur**** (il n'y a pas de serveur).
- * ? ****APIs publiques officielles**** (BAN, SIRENE, gouvernementales).

- * ? **Cle API ORS stockée localement**, jamais committée.
- * ? **Photos OCR traitées localement** par Google ML Kit (sur appareil, pas en cloud).
- * ? **Privacy policy + CGU** rédigées et embarquées dans l'app.
- * ?? **Pas de chiffrement** de la base SQLite locale. Si le téléphone est volé et déverrouillé, le carnet d'adresses + toutes les tournées sont lisibles. Acceptable pour un usage perso, à reconsidérer pour un usage pro (B2B).
- * ?? **Pas de mécanisme de suppression** automatique des anciennes tournées : la base grossit sans limite. À surveiller après 1+ an d'usage.

8. Roadmap restante

À faire avant publication Play Store

1. **Génération keystore** + signing test (docs/keystore-release.md` est prêt à l'emploi)
2. **Screenshots Play Store** (8 captures recommandées : home, tournée optimisée, bottom sheet d'arrêt, carte, scan bordereau, stats, carnet, paramètres)
3. **Compte développeur Google Play** (25 \$ unique -- seule étape qui demande une CB)
4. **Test sur 2-3 autres modèles Android** que le Xiaomi de Noah (idéalement Samsung Galaxy entrée de gamme + tablette)
5. Description Play Store + mots-clés

Améliorations terrain (sans CB)

- * **Mode hors-ligne basique** (cache tiles + queue de géocodage offline)
- * **Scan bordereaux collés sur colis** (en attente des photos terrain de Noah)
- * **Mode sombre** (gros refactor mais ergonomique)
- * Édition rapide notes d'arrêt depuis bottom sheet (extension de B2)
- * Réordonnancement en mode "tournée en cours" (déjà OK techniquement, valider UX)

Plus loin (CB ou gros chantier)

- * **GPS turn-by-turn intégré** (Mapbox SDK ou OSRM custom -- gros)
- * **Crash reporting** (Sentry self-hosted possible mais lourd, Firebase requiert CB)
- * **Multi-langue anglais** (flutter_localizations`, simple mais Noah utilise seul en FR)

9. Recommandations classées

Rang	Action	Effort	Impact
1	Générer la keystore + signer un AAB	30 min	* débloque publication
2	Tester sur 2 autres appareils Android	1 h	* réduit risque crash terrain
3	Mode hors-ligne basique (cache tiles seulement)	2-3 h	* confort terrain rural
4	Captures Play Store + description	2 h	* livrable publication
5	Compte Play Console (25 \$) + publication	1 h	* dépend des 4 ci-dessus
6	Scan bordereaux collés (1er format à tester)	4 h	* confort scan

7	Mode sombre	4-6 h	* confort UX
8	GPS turn-by-turn intégré	20+ h	* gros confort (mais lourd)

* = critique, * = important, * = confort

10. Conclusion

opti_route est une app ****fonctionnellement complète**** pour un usage quotidien de livreur. Tout ce qui était dans la roadmap "version free MVP" est livré sauf 3 features volontairement reportées (mode sombre, hors-ligne, scan colis).

Le code est propre (0 issue analyzer, 0 TODO, structure 3 couches respectée), bien testé sur les zones critiques (85 tests), et documenté (journal de bord PDF + procédure keystore + mentions légales).

****Prochaine étape recommandée**** : générer la keystore release et soumettre une 1ère version sur le Play Store en accès interne (? 100 testeurs). Permet de valider le flow de publication sans l'enjeu d'un audit public, et de détecter les bugs de stack debug -> release qu'on n'aurait pas vus en local.